Curso de Ciência da Computação
Disciplina: Análise e Projeto de Sistemas

Sistema de Controle de Portaria

Documentação técnica e acadêmica

Análise, requisitos, modelagem UML e verificação do sistema implementado

Kayque da Silva Cremonini
Felipe Rafael Dias Delfante

Versão 1.0.0 do sistema (commit f9a22ea)
Novembro de 2026

https://github.com/Kaycoala/Projeto_Portaria

Sumário

Sumário 2

1. Contextualização 3

2. Problema identificado 3

3. Objetivos 3

4. Escopo 4

5. Método de análise e evolução do projeto 5

6. Arquitetura e tecnologias 7

7. Perfis de usuário e permissões 7

8. Requisitos 9

9. Diagrama de casos de uso 11

10. Diagrama de classes e modelo de dados 14

11. Verificação realizada 16

12. Divergências, limitações e pontos não verificados 16

13. Conclusão 17

1. Contextualização

A portaria é o ponto em que um condomínio, uma empresa ou outro ambiente controlado decide quem entra, quando entra e com quem vai falar. Quando esse controle é feito em caderno ou planilha, o registro depende da letra, da pressa e da memória de quem está de plantão: horários ficam em branco, visitas entram sem que o morador tenha sido consultado e, passados alguns dias, ninguém consegue dizer com segurança quem estava no local em determinado momento.

O **Sistema de Controle de Portaria** foi desenvolvido na disciplina de Análise e Projeto de Sistemas para organizar esse processo. Ele é uma aplicação web usada pelo navegador, escrita em Python com Flask, que guarda os dados em um banco SQLite e reúne, em um único lugar, o cadastro de pessoas e veículos, a solicitação e a decisão de autorização de visitantes, o registro de entradas e saídas, as ocorrências, o histórico de acessos e os relatórios. Cada usuário enxerga apenas o que o seu perfil permite.

Este documento descreve o sistema **como ele está implementado** no repositório, na versão 1.0.0. Para isso, o código, o banco de dados, o histórico de commits e os documentos anteriores foram analisados e comparados com os requisitos originais (seção 5 e seção 8). Quando o comportamento real do sistema difere do que o README ou os requisitos afirmam, a diferença está registrada na seção 12.

2. Problema identificado

Os problemas abaixo justificam o sistema e todos têm uma funcionalidade correspondente no código:

- **Entradas e saídas sem registro confiável.** Sem um registro automático, o horário é anotado à mão. O sistema grava data e hora no momento do registro (o usuário não digita o horário) e mostra, na tela da portaria, quem ainda está dentro.
- **Inconsistências nos registros manuais.** Pessoas com o mesmo CPF e veículos com a mesma placa duplicados. O banco recusa CPF de pessoa e placa de veículo repetidos.
- **Dificuldade de localizar o histórico.** A tela de histórico pesquisa por nome, CPF, placa, período e tipo de acesso.
- **Controle de visitantes e autorização de acesso.** Todo visitante nasce com uma solicitação pendente; a entrada só é liberada depois que o aprovador autoriza, dentro do período definido.
- **Acompanhamento de veículos e ocorrências.** Veículos têm proprietário e entram e saem em registros próprios; ocorrências guardam data, horário, descrição e pessoas envolvidas.
- **Preservação das informações.** Registros de acesso e ocorrências não são apagados do banco; a exclusão é lógica e restrita ao administrador.

3. Objetivos

3.1 Objetivo geral

Oferecer um sistema web funcional que organize o controle de entrada e saída de pessoas e veículos em uma portaria, com autorização de visitantes, histórico consultável e controle de acesso por perfil de usuário.

3.2 Objetivos específicos

- Cadastrar moradores, funcionários, prestadores e veículos, sem duplicidade de CPF e de placa.
- Cadastrar visitantes e registrar a decisão do aprovador (autorizar com período ou negar).
- Registrar entradas e saídas de pessoas, visitantes e veículos com data e hora automáticas.

- Impedir a entrada de visitante sem autorização válida para o dia.
- Consultar o histórico de acessos com filtros e registrar ocorrências.
- Gerar relatórios por período, com exportação em CSV, e apresentar um painel com os números atuais.
- Controlar o acesso às funções por perfil (administrador, porteiro, aprovador e funcionário) e exigir login.
- Manter cópias de segurança do banco de dados.

4. Escopo

4.1 O que o sistema contempla

Os dez requisitos funcionais (RF-01 a RF-10), os cinco requisitos não funcionais (RNF-01 a RNF-05) e as dez regras de negócio (RN-01 a RN-10), no grau de atendimento indicado na seção 8. O sistema roda em um computador e é acessado pelo navegador.

4.2 O que está fora do escopo

- Acesso de visitantes ao sistema: o visitante é apenas um cadastro, sem login.
- Veículos de visitantes: todo veículo precisa de uma pessoa cadastrada como proprietária.
- Integração com catracas, câmeras, leitores de placa ou outros equipamentos.
- Aplicativo para celular, envio de mensagens e notificações a moradores.
- Agendamento automático do backup pelo próprio sistema (depende do Agendador de Tarefas do Windows, seção 8.2).
- Troca de senha pelo próprio usuário: a senha é definida ou redefinida pelo administrador na tela de usuários.

5. Método de análise e evolução do projeto

5.1 O que foi examinado

Foi clonado o repositório (branch main, commit f9a22ea) e examinados: o README e o CHANGELOG; o histórico completo de commits; a estrutura de pastas; o esquema do banco (*schema.sql*); todas as rotas, serviços e modelos de página; o arquivo de permissões; o documento de requisitos (*requisitos do sistema.docx*); os casos de uso textuais em PDF; os dois diagramas UML em JPG e os arquivos editáveis *.drawio*; e os 262 testes automatizados. Além da leitura, foram feitas três verificações práticas: execução da suíte de testes, verificação de cada rota em cada perfil e medição de tempo de resposta com 30 mil acessos (seção 11). Informações que não puderam ser comprovadas estão marcadas como tal.

5.2 Evolução por versão

A tabela cruza o roteiro do README com os commits reais. A coluna de testes conta os métodos de teste existentes no commit indicado.

Versão	Commit e data	O que foi comprovado no código	Requisitos tratados	Testes
0.1.0	cd2c859 04/11/2026	Aplicação Flask mínima: create_app, página inicial, base.html, run.py e requirements.txt, dentro da pasta sistema/.	Base do projeto	0
0.2.0	ec93f6e 04/11/2026	Banco SQLite: schema.sql com 9 tabelas e 4 índices, dados_iniciais.sql (4 perfis), db.py e criar_banco.py.	Fundamento de RN-02, RN-04, RN-06, RN-07, RN-08, RN-10 (restrições UNIQUE, CHECK e campo ativo)	14
0.3.0	30ee5ab 04/11/2026	Login com senha em hash (pbkdf2:sha256), proteção CSRF, gerenciamento de usuários, permissoes.py e criar_admin.py.	RNF-02, RNF-04	42
0.4.0	95f8b61 04/11/2026	Cadastro, busca, edição e desativação de pessoas e veículos, com validação de CPF e placa.	RF-01, RF-02, RN-06, RN-07	82
0.5.0	72d4e8b 04/11/2026	Cadastro de visitantes que já abre solicitação pendente; tela do aprovador para autorizar (com período) ou negar.	RF-05, RF-06, RN-09	114
0.6.0	426e213 04/11/2026	Tela da portaria: entrada e saída de pessoas, visitantes e veículos; bloqueio de visitante sem autorização; lista Dentro agora.	RF-03, RF-04, RF-07, RN-01, RN-02, RN-03, RN-04	146
0.7.0	6de7e0c 04/11/2026	Histórico com filtros; edição da observação e exclusão lógica pelo administrador; ocorrências com pessoas envolvidas.	RF-08, RF-09, RN-05, RN-08, RN-10	189
0.8.0	557e3a4 11/11/2026 (tag v0.8.0)	Painel inicial com números do banco; relatório por período com resumo, CSV e impressão; permissão ver_painel; .gitignore.	RF-10	214
0.9.0	9187177 12/11/2026	Backup do banco (serviço, script backup.py, tela e aviso após 24 h); menu lateral responsivo; JavaScript de apoio; permissão gerenciar_backup.	RNF-05 (parcial)	257
1.0.0	f9a22ea 12/11/2026	popular_demo.py (dados fictícios), roteiros de prints e vídeo, gerar_pdf.py, Correspondencia_Modelagem.md; modo de depuração desligado por padrão; correção do CSV.	Estabilização e documentação	262

5.3 Mudanças relevantes para a arquitetura

- A arquitetura em camadas (rotas, serviços, banco) foi definida na versão 0.3.0 e mantida até a 1.0.0. As regras de negócio ficam em *app/services*, as telas em *app/routes* e *app/templates*.

- O esquema do banco foi criado por inteiro na versão 0.2.0 e não foi alterado depois (o histórico do arquivo *schema.sql* tem um único commit). Todas as funcionalidades seguintes usaram as tabelas já previstas.
- O arquivo de permissões nasceu na 0.3.0 com 9 permissões e ganhou *ver_painel* (0.8.0) e *gerenciar_backup* (0.9.0), chegando às 11 permissões atuais.

5.4 Observações sobre o histórico do GitHub

- Existe uma única tag no repositório, *v0.8.0*. As demais versões são identificadas pelas mensagens de commit; a versão 0.9.0 não tem commit com esse rótulo: seu conteúdo está no commit "feat: adiciona backup do banco de dados".
- As versões 0.1.0 a 0.7.0 foram commitadas no mesmo dia (04/11/2026), por Felipe Rafael. A fase de requisitos e diagramas UML é anterior, de 28/10/2026, em commits próprios; o commit da 0.1.0 só acrescenta a pasta *sistema/*.
- A versão 0.8.0 teve duas tentativas anteriores (commits c280dc9 e 03c8977) com arquivos soltos na raiz do repositório, removidos em seguida; a versão correta está no commit 557e3a4.
- Um commit de melhoria de interface (05a3288) foi revertido (caa885e). Os recursos de interface que constam do CHANGELOG da 0.9.0 (menu responsivo, scripts) estão no commit do backup e foram confirmados nos arquivos.

6. Arquitetura e tecnologias

O sistema usa **Python e Flask** no servidor, **SQLite** (módulo sqlite3 da biblioteca padrão) como banco, páginas **Jinja** com **Bootstrap 5.3.3** e testes com **unittest**. Um único computador executa a aplicação, sem depender de servidor de banco de dados.

Camada	Pasta ou arquivo	Responsabilidade
Apresentação	sistema/app/templates, static	Páginas HTML, menu lateral que mostra só o que o perfil pode usar, CSS e JavaScript.
Rotas	sistema/app/routes (11 módulos)	Recebem as requisições, conferem a permissão e chamam os serviços.
Regras de negócio	sistema/app/services	Validações, regras RN, consultas e gravações (um serviço por assunto).
Segurança	auth.py, permissoes.py, seguranca.py	Login, sessão, permissões por perfil, proteção CSRF.
Dados	sistema/banco, db.py	schema.sql, dados iniciais e conexão com chaves estrangeiras ligadas.
Utilitários	criar_admin.py, backup.py, popular_demo.py	Primeiro administrador, backup do banco, dados fictícios de demonstração.
Testes	sistema/tests (11 arquivos)	262 testes automatizados.

O banco tem nove tabelas: *perfil, pessoa, usuario, veiculo, visitante, autorizacao, registro_acesso, ocorrencia* e *ocorrencia_pessoa*. O vínculo do registro de acesso com a pessoa, o visitante ou o veículo é garantido por uma restrição CHECK: conforme o tipo, um desses três campos é obrigatório. A saída não pode ser anterior à entrada.

7. Perfis de usuário e permissões

O sistema define quatro perfis, cadastrados em *dados_iniciais.sql* e usados em *permissoes.py*. Cada permissão lista quais perfis podem exercê-la; a tabela abaixo foi conferida por execução, solicitando cada página e cada ação com um usuário de cada perfil (resposta 403 quando não permitido).

Permissão (nome no código)	Adm.	Port.	Aprov.	Func.
Ver painel (ver_painel)	sim	sim	sim	não
Gerenciar usuários (gerenciar_usuarios)	sim	não	não	não
Cadastrar e editar pessoas e veículos (gerenciar_cadastros)	sim	sim	não	não
Cadastrar visitantes e nova solicitação (cadastrar_visitantes)	sim	sim	não	não
Registrar entradas e saídas (registrar_acesso)	sim	sim	não	não
Autorizar ou negar visitantes (autorizar_visitantes)	sim	não	sim	não
Consultar histórico de acessos (consultar_historico)	sim	sim	sim	não
Registrar ocorrências (registrar_ocorrencia)	sim	sim	não	não
Corrigir ou excluir registros (administrar_registros)	sim	não	não	não
Gerar relatórios (ver_relatorios)	sim	não	não	não
Backup do banco (gerenciar_backup)	sim	não	não	não

Perfil	Responsabilidades confirmadas no código
Administrador	Tem as 11 permissões. Além das funções do porteiro e do aprovador, gerencia usuários, gera relatórios e faz backup, e é o único que corrige a observação de um acesso, exclui acessos e ocorrências e edita visitantes e ocorrências.
Porteiro	Cadastra pessoas, veículos e visitantes, registra entradas e saídas, registra ocorrências, consulta o histórico e vê o painel. Não autoriza visitantes, não gera relatórios e não altera nem exclui registros.

Perfil	Responsabilidades confirmadas no código
Aprovador (Gerente)	Autoriza (com período) ou nega solicitações de visitantes, consulta o histórico e vê o painel. Não registra entradas nem ocorrências.
Funcionário	Consegue entrar no sistema, mas não tem nenhuma permissão: a página inicial exibe que o perfil não tem operações. A descrição do perfil no banco ("tem seus dados e veículos cadastrados") refere-se ao fato de a pessoa poder ser cadastrada, e não a uma função do sistema.
Visitante	**Não é usuário do sistema.** É um registro (tabela visitante) criado pelo porteiro, sem login nem senha. A tabela de usuários só se liga opcionalmente a uma pessoa cadastrada, nunca a um visitante.

8. Requisitos

Os identificadores originais foram preservados. A coluna *Situação* usa quatro valores: **Implementado**, **Implementado com ressalva** (atende, com diferença em relação ao texto original), **Parcial** e **Não comprovado**. A coluna *Evidência* aponta onde o comportamento está no código.

8.1 Requisitos funcionais

ID	Descrição revisada	Situação	Evidência e observações
RF-01	O sistema deve permitir que o porteiro ou o administrador cadastrem, consultem, editem e desativem pessoas (morador, funcionário ou prestador), informando nome, CPF, telefone, e-mail e endereço ou setor.	Implementado	pessoa_service.py; menu Pessoas. Desativar não apaga, para preservar histórico.
RF-02	O sistema deve permitir cadastrar, consultar, editar e desativar veículos, informando placa, modelo, cor e a pessoa proprietária (cadastrada e ativa).	Implementado	veiculo_service.py; menu Veículos. Aceita placa no formato antigo e Mercosul.
RF-03	O sistema deve permitir registrar a entrada de pessoas cadastradas, de visitantes autorizados e de veículos, gravando automaticamente data e horário e o porteiro responsável.	Implementado	acesso_service.registrar_entrada_*; menu Entrada e saída. Recusa quem já está dentro.
RF-04	O sistema deve permitir registrar a saída de quem tem entrada sem saída, gravando automaticamente data e horário.	Implementado	acesso_service.registrar_saida. O horário é do sistema; o usuário não o digita (o texto original dizia "informando").
RF-05	O sistema deve permitir que o porteiro cadastre visitantes (nome, CPF, telefone, pessoa visitada e motivo), abrindo junto uma solicitação de autorização pendente.	Implementado	visitante_service.criar_visitante grava visitante e autorização na mesma transação. Edição só pelo administrador.
RF-06	O sistema deve permitir que o aprovador (ou o administrador) autorize, informando a data inicial e, opcionalmente, a final, ou negue a entrada de um visitante, registrando quem decidiu e quando.	Implementado com ressalva	autorizacao_service.aprovar/negar. O texto original citava "morador ou responsável"; no sistema decide o perfil Aprovador (ou Administrador) e não há vínculo entre a pessoa visitada e quem decide. A decisão não pode ser desfeita.
RF-07	O sistema deve permitir registrar a entrada e a saída de veículos, com placa, horário e proprietário responsável.	Parcial	Implementado para veículos de pessoas cadastradas, em registro próprio (o registro do veículo é independente do da pessoa). O original previa também "visitante responsável": não é possível, pois veículo exige proprietário na tabela de pessoas.
RF-08	O sistema deve permitir que porteiro, aprovador e administrador consultem o histórico de acessos filtrando por nome, CPF, placa, período e tipo.	Implementado	historico_service.listar_historico. Mostra até 200 registros; o administrador também vê os excluídos, marcados.
RF-09	O sistema deve permitir que o porteiro e o administrador registrem ocorrências com data, horário, descrição (5 a 2.000 caracteres) e pessoas envolvidas (opcional).	Implementado	ocorrencia_service.py. Não aceita data futura.
RF-10	O sistema deve permitir que o administrador gere relatório por período (até 366 dias) com entradas por tipo, saídas, visitantes, veículos, autorizações e ocorrências, com exportação em CSV e impressão.	Implementado	relatorio_service.py; menu Relatórios (somente administrador).

8.2 Requisitos não funcionais

ID	Descrição revisada	Situação	Evidência e observações
RNF-01	As consultas e os registros de acesso devem responder em até 2 segundos.	Implementado (medido)	Índices no banco, limite de 200 linhas no histórico e relatório de no máximo 366 dias. Medido nesta análise com 30.000 acessos: maior tempo 0,56 s (seção 11). Não existe teste automatizado de desempenho no repositório.
RNF-02	O acesso ao sistema deve exigir usuário e senha.	Implementado	auth.py bloqueia todas as rotas, exceto login e arquivos estáticos; senha em hash pbkdf2:sha256; token CSRF em todo formulário. Senha mínima de 6 caracteres.
RNF-03	O sistema deve permanecer disponível durante o funcionamento da portaria.	Não comprovado	Nada no código ou nos testes garante disponibilidade; depende do computador e de como o sistema é mantido em execução. O README apresenta apenas a escolha de uma solução leve que roda localmente.
RNF-04	Cada usuário deve acessar apenas as funções do seu perfil.	Implementado	permissoes.py (11 permissões) e decorador exige_permissao; páginas proibidas respondem 403 e o menu esconde o que não é permitido. Verificado por execução.
RNF-05	O sistema deve permitir cópia de segurança dos dados com intervalo máximo de 24 horas entre elas.	Parcial	backup_service.py, backup.py e tela Backup: cópia consistente do SQLite, conferida por integrity_check, com retenção de 30 cópias e aviso no painel do administrador quando passa de 24 horas. O sistema não agenda a execução diária; isso depende de configurar o Agendador de Tarefas (instruções no README do sistema). A configuração não é verificável pelo repositório.

8.3 Regras de negócio

ID	Descrição revisada	Situação	Evidência e observações
RN-01	Só pessoas cadastradas e ativas podem ter a entrada registrada.	Implementado	registrar_entrada_pessoa recusa pessoa inexistente ou inativa.
RN-02	Toda entrada deve ter data e horário, gravados pelo sistema no momento do registro.	Implementado	Coluna entrada_em NOT NULL, preenchida com datetime('now','localtime').
RN-03	O visitante só entra com autorização aprovada e vigente na data da entrada.	Implementado com ressalva	conferir_autorizacao_do_visitante. O texto original admitia também a "liberação pelo responsável da portaria"; no sistema o porteiro não libera por conta própria, só a autorização aprovada vale.
RN-04	A saída de quem entrou deve ser registrada.	Implementado com ressalva	Quem não saiu permanece em Dentro agora e no painel. O sistema não obriga nem alerta sobre saída esquecida.
RN-05	Só o administrador altera ou exclui registros de acesso, visitantes e ocorrências.	Implementado com ressalva	Permissão administrar_registros. No acesso só se corrige a observação (horários e vínculos não mudam); não existe exclusão de visitante; autorizações decididas não são alteráveis. O porteiro edita pessoas e veículos, que a regra não cita.
RN-06	Não podem existir duas pessoas com o mesmo CPF.	Implementado	Validação e UNIQUE em pessoa.cpf. O sistema exige 11 números, mas não confere os dígitos verificadores. O CPF de visitante não é único.
RN-07	Não podem existir dois veículos com a mesma placa.	Implementado	Validação do formato (antigo ou Mercosul) e UNIQUE em veiculo.placa.
RN-08	Registros de acesso não são excluídos pelo usuário comum; permanecem para consulta do administrador.	Implementado	Exclusão lógica (ativo = 0), visível apenas ao administrador. O mesmo vale para ocorrências.
RN-09	Visitante sem autorização prévia aguarda a decisão e não pode ter entrada registrada.	Implementado	Autorização nasce como pendente; a portaria recusa a entrada enquanto pendente ou negada.
RN-10	Toda ocorrência tem data, horário e descrição.	Implementado	data_hora e descricao NOT NULL, descrição não vazia (CHECK) e com mínimo de 5 caracteres.

9. Diagrama de casos de uso

O diagrama original (*diagramas/Diagrama_Casos_de_Uso_Portaria.jpg*) foi comparado com o código e corrigido. A figura a seguir é a versão revisada.

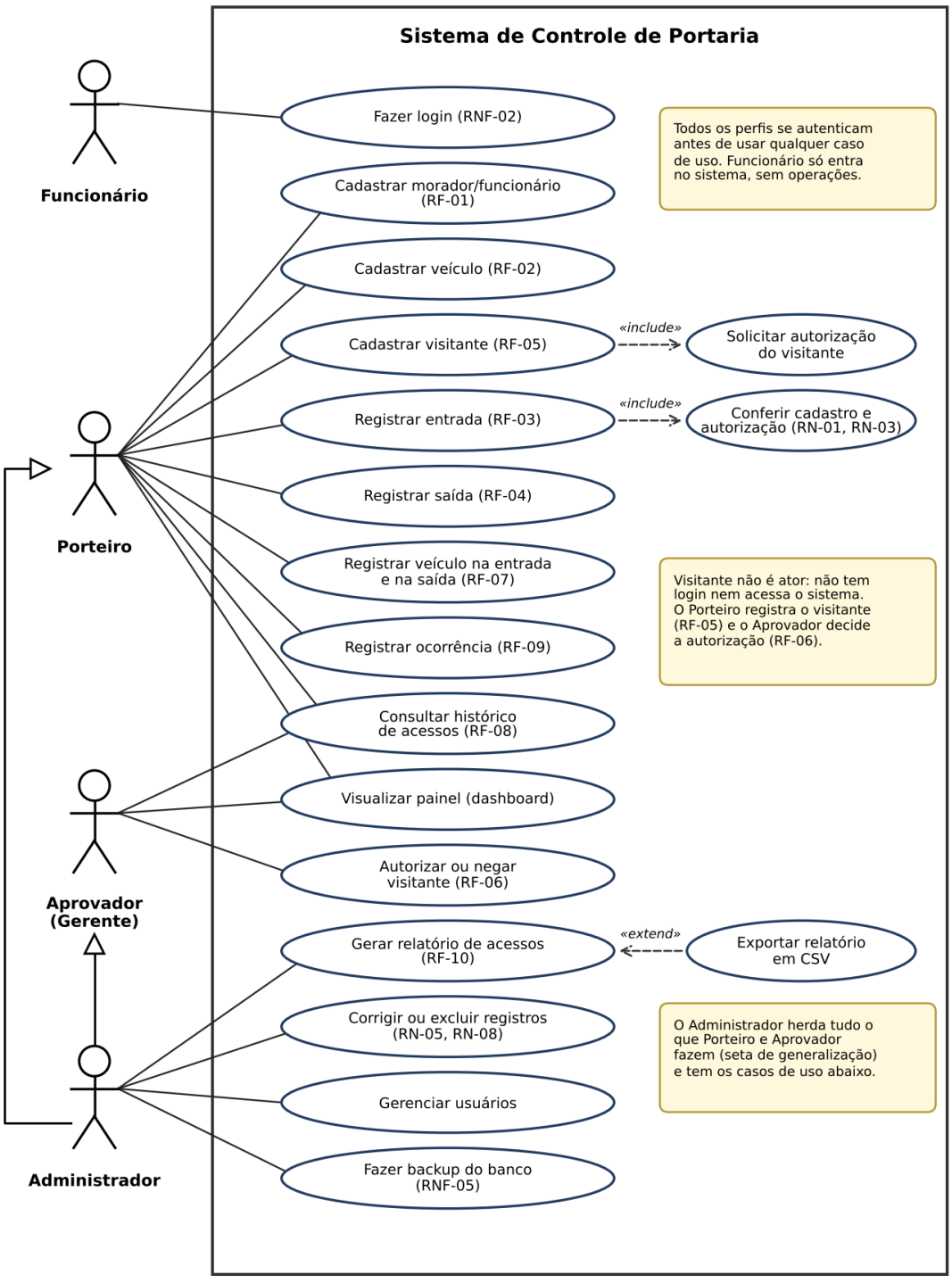


Figura 1 - Diagrama de casos de uso revisado, coerente com as permissões implementadas.

9.1 O que foi corrigido em relação ao diagrama original

Problema no diagrama original	Situação no sistema	Correção
O Funcionário aparece ligado a "Cadastrar morador/funcionário" e "Cadastrar veículo".	Quem cadastra é o porteiro (e o administrador). O funcionário só entra no sistema e não tem operações.	Cadastros ligados ao Porteiro; Funcionário ligado apenas a Fazer login.
O Visitante é um ator, com os casos "Solicitar acesso" e "Aguardar autorização".	O visitante não tem login nem usa o sistema; o porteiro o cadastra e a solicitação é aberta junto.	Visitante removido como ator, explicado em nota; "Solicitar autorização" vira caso incluído em Cadastrar visitante.
O Administrador aparece ligado só ao histórico, às ocorrências e ao relatório.	Tem todas as permissões e executa também usuários, backup e correção de registros.	Generalização: o Administrador herda Porteiro e Aprovador e ganha os casos exclusivos.
Faltam casos de uso existentes no sistema.	Existem login, painel, gerenciamento de usuários, backup e correção ou exclusão lógica de registros.	Casos acrescentados.
Incluídos "Identificar pessoa ou veículo" e "Identificar registro de acesso", que são apenas passos de tela.	A regra real é conferir cadastro e, para visitante, a autorização válida (RN-01, RN-03).	Substituídos por "Conferir cadastro e autorização".
O Aprovador não aparece com acesso ao histórico e ao painel.	O Aprovador tem consultar_historico e ver_painel.	Ligações acrescentadas.

9.2 Casos de uso textuais

Os três casos de uso do documento original foram mantidos e atualizados com o comportamento real do sistema.

UC01 - Registrar entrada (RF-03, RF-07)

Ator principal: Porteiro (o Administrador também pode)

Objetivo: Registrar a entrada de uma pessoa, de um visitante ou de um veículo.

Pré-condições:

- O usuário está autenticado e tem a permissão de registrar acessos.
- Pessoa e veículo estão cadastrados e ativos; o visitante está cadastrado.

Fluxo principal:

1. O porteiro abre a tela Entrada e saída.
2. O sistema lista pessoas, visitantes e veículos que ainda não estão dentro.
3. O porteiro escolhe quem está entrando e, se quiser, escreve uma observação.
4. O porteiro aciona Registrar entrada.
5. O sistema confere o cadastro ativo (RN-01) e, para visitante, a autorização válida na data de hoje (RN-03, RN-09).
6. O sistema grava a entrada com data, horário e porteiro (RN-02).
7. O sistema confirma o registro e o item passa a aparecer em Dentro agora.

Fluxos alternativos:

- Visitante com autorização pendente, negada, ainda não iniciada ou vencida: a entrada é bloqueada e a mensagem informa o motivo.
- Pessoa ou veículo inativo, ou veículo de proprietário inativo: o sistema recusa a entrada.
- Quem já está dentro: o sistema recusa e pede a saída antes de uma nova entrada.
- Pessoa não cadastrada: não aparece na lista; é preciso cadastrá-la antes (RN-01).

Pós-condição: A entrada fica no histórico de acessos e na lista Dentro agora.

UC02 - Autorizar visitante (RF-06)

Ator principal: Aprovador (Gerente); o Administrador também pode

Objetivo: Autorizar, com período, ou negar a entrada de um visitante.

Pré-condições:

- O aprovador está autenticado.
- Existe uma solicitação pendente (criada no cadastro do visitante ou em Nova solicitação).

Fluxo principal:

1. O aprovador abre Autorizações, que mostra as pendentes.
2. O aprovador escolhe Analisar em uma solicitação.
3. O sistema mostra visitante, CPF, telefone, pessoa visitada e motivo.
4. O aprovador informa a data inicial (padrão: hoje), a data final (opcional) e uma observação.
5. O aprovador escolhe Autorizar.
6. O sistema valida o período e registra a decisão com o aprovador e o momento.
7. O sistema confirma que o visitante está autorizado.

Fluxos alternativos:

- O aprovador escolhe Negar: o sistema registra a negativa; a entrada do visitante fica bloqueada.
- Data final anterior à inicial, ou período já encerrado: o sistema recusa e pede a correção.
- Solicitação já decidida (inclusive por outro aprovador ao mesmo tempo): o sistema recusa a alteração.

Pós-condição: A decisão fica registrada e não pode ser desfeita. Se o visitante voltar em outro dia, o porteiro abre uma nova solicitação.

UC03 - Consultar histórico de acessos (RF-08)

Ator principal: Porteiro, Aprovador ou Administrador

Objetivo: Consultar os registros de entrada e saída de pessoas, visitantes e veículos.

Pré-condições:

- O usuário está autenticado e tem a permissão de consultar o histórico.

Fluxo principal:

1. O usuário abre Histórico.
2. O sistema mostra os filtros: nome, CPF, placa, data inicial, data final e tipo.
3. O usuário informa zero ou mais filtros e aciona Pesquisar.
4. O sistema pesquisa os registros ativos que atendem aos critérios.
5. O sistema apresenta até 200 resultados, do mais recente para o mais antigo.

Fluxos alternativos:

- Nenhum registro encontrado: o sistema informa que não há resultados para os filtros.
- Data inválida ou CPF sem números: o sistema mostra a mensagem de erro.
- Mais de 200 resultados: o sistema avisa que mostra os mais recentes; os filtros refinam a busca.

Pós-condição: Nada é alterado. Apenas o administrador vê os registros excluídos (marcados) e os botões Editar e Excluir.

10. Diagrama de classes e modelo de dados

O diagrama de classes original foi mantido como referência histórica, mas não descreve mais o sistema com fidelidade. A figura abaixo apresenta o modelo implementado, derivado do *schema.sql* e dos serviços.

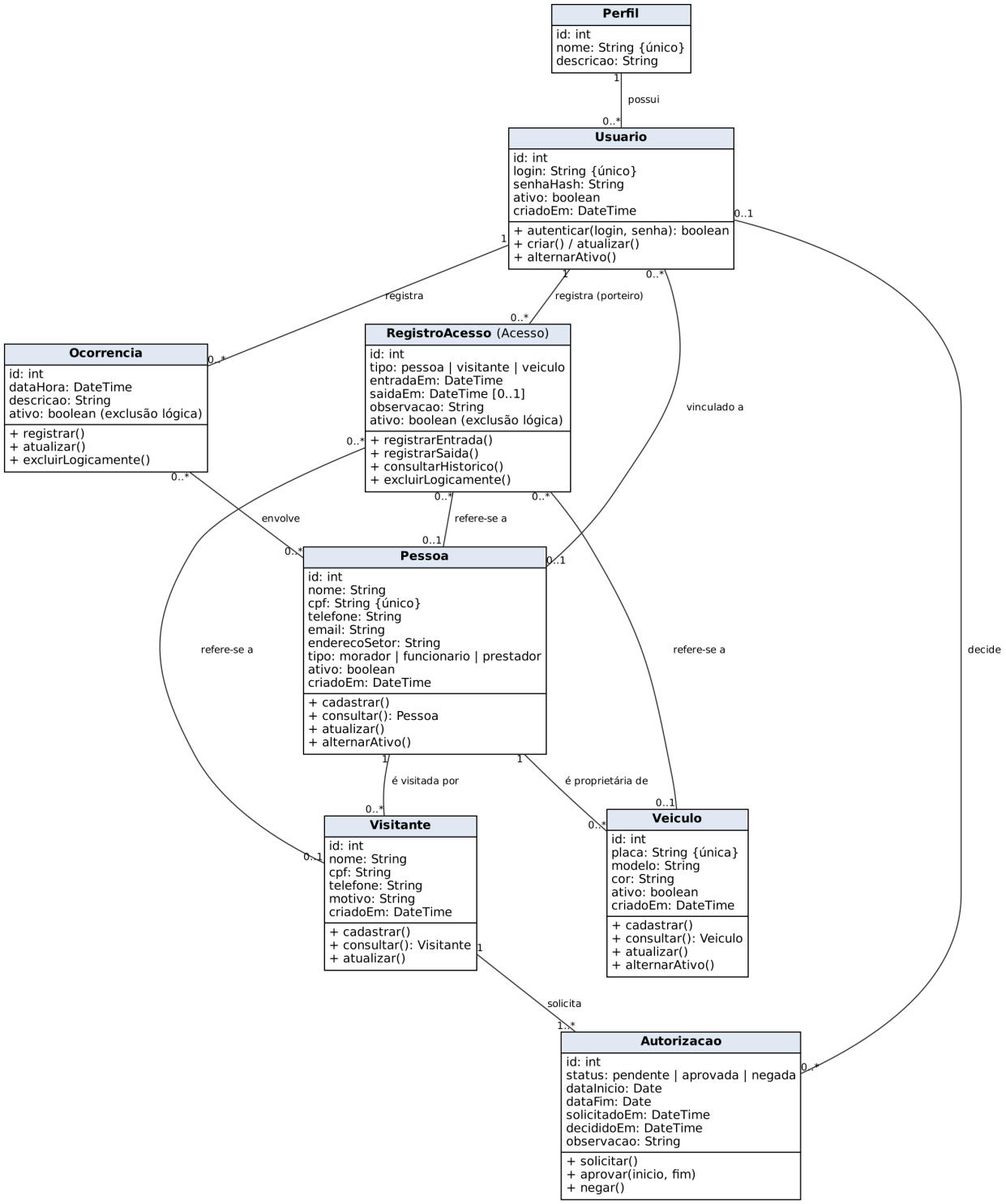


Figura 2 - Diagrama de classes do modelo implementado.

10.1 Diferenças em relação ao diagrama de classes original

No diagrama original	No sistema implementado
Visitante é especialização de Pessoa ("pode ser um").	Visitante é uma tabela independente, com nome, CPF e telefone próprios; liga-se a uma Pessoa apenas como pessoa visitada.
Visitante tem o atributo pessoaVisitada do tipo texto.	É uma chave estrangeira para Pessoa (pessoa_visitada_id), obrigatória.
Acesso se relaciona apenas com Pessoa.	O registro de acesso se relaciona com Pessoa, Visitante ou Veículo, conforme o tipo, e com o Usuário que registrou.
Acesso "pode registrar" Ocorrência.	A ocorrência não se liga a um acesso: liga-se ao usuário que a registrou e, por associação N:N, às pessoas envolvidas.
Não existe classe para a decisão de autorização.	Existe Autorizacao (status, período, aprovador, datas), indispensável para RF-06 e RN-09.
Usuario tem o atributo perfil.	Perfil é uma classe própria; as permissões de cada perfil ficam em permissoes.py.
Atributos de situação ausentes.	Pessoa, Veiculo, Usuario, RegistroAcesso e Ocorrencia têm o campo ativo, que permite desativar ou excluir logicamente sem perder dados (RN-08).

10.2 Correspondência entre classes, tabelas e serviços

Classe	Tabela	Serviço (operações)	Restrições principais
Perfil	perfil	usuario_service	nome único; 4 perfis iniciais
Usuario	usuario	usuario_service (autenticar, criar, atualizar, alternar_ativo)	login único; senha em hash; não permite desativar o último administrador
Pessoa	pessoa	pessoa_service	CPF único; tipo em (morador, funcionário, prestador)
Veiculo	veiculo	veiculo_service	placa única; proprietário obrigatório
Visitante	visitante	visitante_service	pessoa visitada e motivo obrigatórios; CPF não é único
Autorizacao	autorizacao	autorizacao_service (solicitar_nova, aprovar, negar)	status em (pendente, aprovada, negada)
RegistroAcesso	registro_acesso	acesso_service, historico_service	CHECK por tipo; saída não anterior à entrada; exclusão lógica
Ocorrencia	ocorrencia, ocorrencia_pessoa	ocorrencia_service	data, hora e descrição obrigatórias; exclusão lógica

Observação: o código não usa classes de domínio com métodos. As operações dos diagramas correspondem a funções nos serviços. Por isso, os métodos desenhados nas classes representam a responsabilidade lógica de cada entidade.

11. Verificação realizada

11.1 Testes automatizados

A suíte do projeto foi executada nesta análise: **262 testes, todos aprovados** em 13 segundos, o mesmo número que o README e o CHANGELOG informam. A distribuição por arquivo é: autenticação (28), pessoas e veículos (40), visitantes e autorizações (32), entradas e saídas (32), histórico e ocorrências (43), dashboard (11), relatórios (14), backup (27), interface (16), banco (14) e dados de demonstração (5).

11.2 Permissões por perfil

Em um banco temporário, foram criados um usuário de cada perfil e solicitadas 21 páginas e 10 ações. Os resultados coincidem com a tabela da seção 7: por exemplo, o aprovador recebe 403 na tela da portaria e nos relatórios; o porteiro recebe 403 em autorizações, usuários, backup e ao excluir registros; o funcionário recebe 403 em tudo, exceto na página inicial; e quem não está autenticado é redirecionado ao login.

11.3 Tempo de resposta (RNF-01)

Em um banco temporário com 30.000 registros de acesso, 500 pessoas e 300 veículos, as telas foram solicitadas por um cliente de teste do Flask, sem rede e sem navegador:

Tela	Tempo
Painel inicial	0,018 s
Entrada e saída	0,020 s
Histórico sem filtro	0,019 s
Histórico com nome e período	0,010 s
Histórico por CPF	0,024 s
Pessoas e veículos	0,021 s e 0,014 s
Relatório de 90 dias	0,141 s
Relatório de 366 dias (limite)	0,562 s

Todos ficaram abaixo de 2 segundos. A medição não inclui a rede nem o navegador e depende do computador usado; o resultado indica folga, mas não substitui um teste no ambiente real. O repositório não traz teste automatizado de desempenho; o valor citado no CHANGELOG (30 mil acessos, relatório de 90 dias em cerca de 0,6 s) é compatível com a ordem de grandeza medida aqui, embora o tempo do relatório de 90 dias tenha sido menor nesta execução.

12. Divergências, limitações e pontos não verificados

12.1 Entre o README, os documentos e o código

- O README exibe a versão 1.0.0 e o status "Estável", mas existe somente a tag v0.8.0 e a versão 0.9.0 não tem commit próprio.
- O README ainda marca como pendentes os prints reais, o vídeo de demonstração e o PDF final. Nesta data a pasta de prints contém apenas o README e a pasta do vídeo, apenas o roteiro.
- O arquivo editável *diagramas/Diagrama_de_Classes_Portaria.drawio* está **vazio** (240 bytes, sem nenhum elemento), embora o README o apresente como editável. Só o JPG tem o conteúdo.
- O diagrama de classes original e o *Correspondencia_Modelagem.md* (que afirma que o diagrama "não foi alterado") mantêm a herança Visitante-Pessoa e a ligação Acesso-Ocorrência, que não existem no código (seção 10.1).

- O README descreve o Funcionário como quem "apenas entra no sistema", o que está correto; já a descrição do perfil em *dados_iniciais.sql* sugere um papel que o código não implementa.

12.2 Entre os requisitos e a implementação

- RF-06 e RN-03: o texto original atribui a autorização ao morador ou ao responsável e admite liberação pela portaria; o sistema concentra a decisão no Aprovador e no Administrador e não admite liberação pelo porteiro.
- RF-07: não há veículo de visitante.
- RNF-03 e RNF-05: dependem do ambiente de uso (execução contínua e agendamento do backup), e isso não é comprovável pelo repositório.
- RN-06: o CPF não tem os dígitos verificadores conferidos, e o CPF de visitante pode se repetir em cadastros diferentes.
- RN-04: o sistema não gera alerta para saída esquecida.

12.3 O que não foi verificado

- Não foram tiradas capturas de tela do sistema em execução: o ambiente de análise não carrega o Bootstrap, que o sistema busca em um CDN, e telas sem estilo não representariam o sistema. Os prints reais continuam dependendo de uma execução no computador dos autores.
- O comportamento em navegador, em celular e em tablet (menu responsivo) não foi testado visualmente.

13. Conclusão

O repositório contém um sistema funcional e bem coberto por testes, que atende sem diferenças relevantes oito dos dez requisitos funcionais (RF-01 a RF-05, RF-08, RF-09 e RF-10), com ressalva o RF-06 e de forma parcial o RF-07. Dos requisitos não funcionais, três estão atendidos ou medidos (RNF-01, RNF-02 e RNF-04), um é parcial (RNF-05) e um não é comprovável pelo código (RNF-03). As regras de negócio estão implementadas, algumas com ressalvas descritas na seção 8.3. A evolução por versões é coerente com o roteiro do README, com os ajustes de histórico da seção 5.4. Os diagramas UML originais precisavam de correção para refletir o sistema real; as versões revisadas estão neste documento e acompanham o PDF como arquivos de imagem.

Repositório analisado: https://github.com/Kaycoala/Projeto_Portaria (branch main, commit f9a22ea).